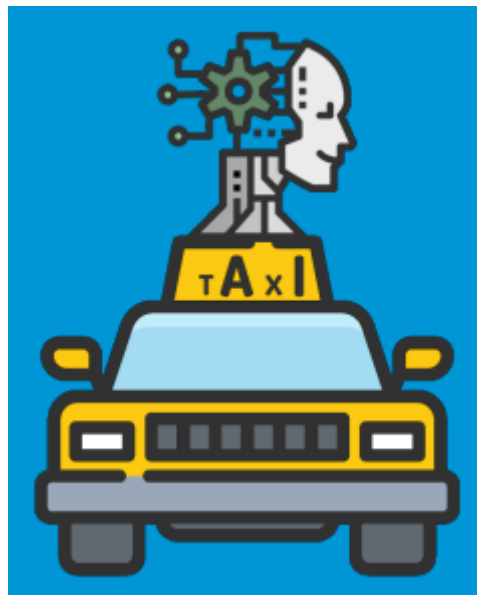


T-AIA-902



Taxi driver



**Dorian AYOUL - Baptiste LEMONNIER -
François PARMENTIER - Pierre HAMEL - Pierre
MAUGER**

Projet et organisation

Dans ce projet, il s'agit de résoudre différents mini-jeux en développant un agent intelligent à l'aide d'un algorithme d'apprentissage par renforcement, tel que le Deep Q-Learning. L'objectif principal est de minimiser le nombre d'étapes nécessaires pour déposer les passagers à leurs destinations.

Nous avons d'abord réalisé un bootstrap sur le jeu Frozen Lake, également issu de la bibliothèque Gymnasium, afin de nous familiariser avec son fonctionnement. Lors de cette étape, nous avons déjà conçu un algorithme capable de résoudre Taxi-v3.

Ayant terminé ce premier objectif, nous avons décidé d'aller plus loin en explorant d'autres environnements Gymnasium. Chaque membre du groupe a ainsi choisi un jeu différent, avec pour mission de développer une IA capable de le résoudre. La répartition était la suivante :

- Dorian sur Bipedal Walker
- Baptiste sur Lunar Lander
- François sur Atari Asterix,
- Pierre M. sur Ant 3D
- Pierre H. sur Car Racing.

Chaque membre était responsable de l'entraînement de son agent et de l'analyse des résultats obtenus.

Apprentissage par renforcement et environnement Gym

Apprentissage par Renforcement

L'Apprentissage par Renforcement (RL) est une branche de l'intelligence artificielle où un agent apprend à prendre des décisions optimales dans un environnement donné, en interagissant avec celui-ci. Contrairement à l'apprentissage supervisé, où l'agent apprend à partir d'exemples étiquetés, le RL fonctionne sur le principe d'essais-erreurs. L'agent effectue des actions, observe les conséquences (récompenses ou punitions), puis ajuste sa stratégie pour maximiser la somme des récompenses à long terme.

Le processus d'apprentissage par renforcement repose sur quatre éléments clés :

- Agent : le décideur ou l'entité qui agit dans l'environnement.
- Environnement : le système ou contexte dans lequel l'agent agit.
- Actions : les choix ou mouvements que l'agent peut effectuer.
- Récompenses (Rewards) : les feedbacks numériques que l'agent reçoit après chaque action, indiquant la qualité de cette action.

L'agent cherche à apprendre une politique (fonction) qui associe chaque état de l'environnement à la meilleure action à prendre, afin d'optimiser la récompense cumulée sur le long terme.

Environnement Gym

Gym est une bibliothèque open source développée par OpenAI, conçue pour faciliter la création, l'évaluation et le test d'algorithmes d'apprentissage par renforcement. Gym fournit un ensemble standardisé d'environnements de simulation, allant des jeux simples aux tâches complexes de robotique.

Chaque environnement Gym respecte une interface simple qui permet à l'agent de :

- Réinitialiser l'environnement (`env.reset()`), pour commencer une nouvelle simulation.
- Effectuer une action (`env.step(action)`), qui retourne l'état suivant, la récompense, un indicateur de fin d'épisode, et des informations supplémentaires.
- Visualiser l'état (`env.render()`), pour observer graphiquement ou visuellement la situation.

Grâce à cette standardisation, Gym facilite la comparaison et le développement d'algorithmes RL, en fournissant un cadre reproductible et modulable.

Bipedal Walker

Bipedal Walker est un environnement de simulation physique mettant en scène un robot bipède doté de quatre articulations (deux hanches et deux genoux). L'objectif est de faire progresser le robot sur un terrain en utilisant des commandes motrices continues.

Modes de jeu

- Normal : terrain légèrement irrégulier,
- Hardcore : environnement complexe incluant fosses et obstacles.

Objectif

Le robot doit atteindre un score de 300 points avant la fin de l'épisode. Celui-ci est atteint si :

- Le torse entre en contact avec le sol,
- Le robot atteint la fin du terrain,
- Le robot atteint la limite d'actions (1600 pour le mode normal et 2000 pour hardcore)

Espace d'action

Le robot reçoit en entrée une commande continue dans l'intervalle $[-1, 1]$ pour chacun de ses 4 moteurs (-1 pour une rotation de l'articulation dans le sens inverse des aiguilles d'une montre, 0 pour aucune rotation et 1 pour le sens des aiguilles d'une montre).

Espace d'observation

Le vecteur d'observation comprend :

- L'angle et la vitesse angulaire du torse,
- Les vitesses horizontale et verticale,
- Les positions et vitesses des articulations (leur sens et force de rotation),
- L'état de contact des jambes avec le sol,
- 10 mesures lidar (laser permettant de détecter les objets) vers l'avant.

Aucune coordonnée absolue n'est incluse (pas de coordonnées de position générale).

Fonction de récompense

- Récompense proportionnelle à l'avancement.
- Pénalité de -100 points en cas de chute.
- Léger coût associé à l'utilisation des moteurs.

Deep Q Learning

Le Q-Learning est un algorithme d'apprentissage par renforcement qui apprend une table de valeurs $Q(s,a)$ pour estimer la récompense attendue d'une action a dans un état s . Cette méthode fonctionne bien dans des environnements à espaces d'états et d'actions discrets et réduits, comme FrozenLake ou Taxi-v3, où toutes les paires (s,a) peuvent être stockées dans une table en mémoire.

Cependant, dans des environnements complexes comme Bipedal Walker, les états (vitesses, positions, capteurs lidar...) et les actions (commandes motrices continues) sont haute dimension et continus, rendant impossible l'utilisation d'une table Q classique.

C'est pourquoi on utilise le Deep Q-Learning (DQN) : une régression fonctionnelle basée sur un réseau de neurones qui approxime la fonction $Q(s,a)$ à partir d'exemples.

Implémentation dans ce projet

Pour adapter DQN à Bipedal Walker, les modifications suivantes ont été apportées :

- Buffer d'expérience : stocke un ensemble d'expériences (s,a,r,s') pour permettre un apprentissage hors ligne et réduire la corrélation entre les données successives.
- Réseaux Actor et Target : le réseau Target est une copie figée (mise à jour périodiquement) du réseau principal (Actor) pour éviter le problème de "moving target" où la cible $Q(s',a')$ change trop rapidement pour être apprise efficacement.
- Discrétisation de l'espace d'action : l'action continue (valeurs décimales infinies entre -1 et 1 pour chaque moteur) a été discrétisée pour s'adapter au cadre du DQN, qui nécessite un espace d'action discret (valeurs fixes, -1, 0 ou 1).

Avantages

- Apprentissage efficace sans table Q : le réseau apprend à estimer les meilleures actions dans des états complexes.
- Expérience réutilisable : grâce au buffer, l'agent peut apprendre à partir de ses anciennes expériences, améliorant la stabilité.

Inconvénients

- Discrétisation sous-optimale : transformer des actions continues en un petit nombre de combinaisons discrètes réduit la finesse des mouvements, ce qui nuit à la qualité de la marche.
- Tuning difficile : choisir un bon découpage de l'espace d'action, la taille du réseau, le taux d'apprentissage, etc., demande beaucoup d'expérimentations.
- Problèmes de convergence : même avec buffers et target networks, l'apprentissage peut rester instable dans un environnement aussi complexe.

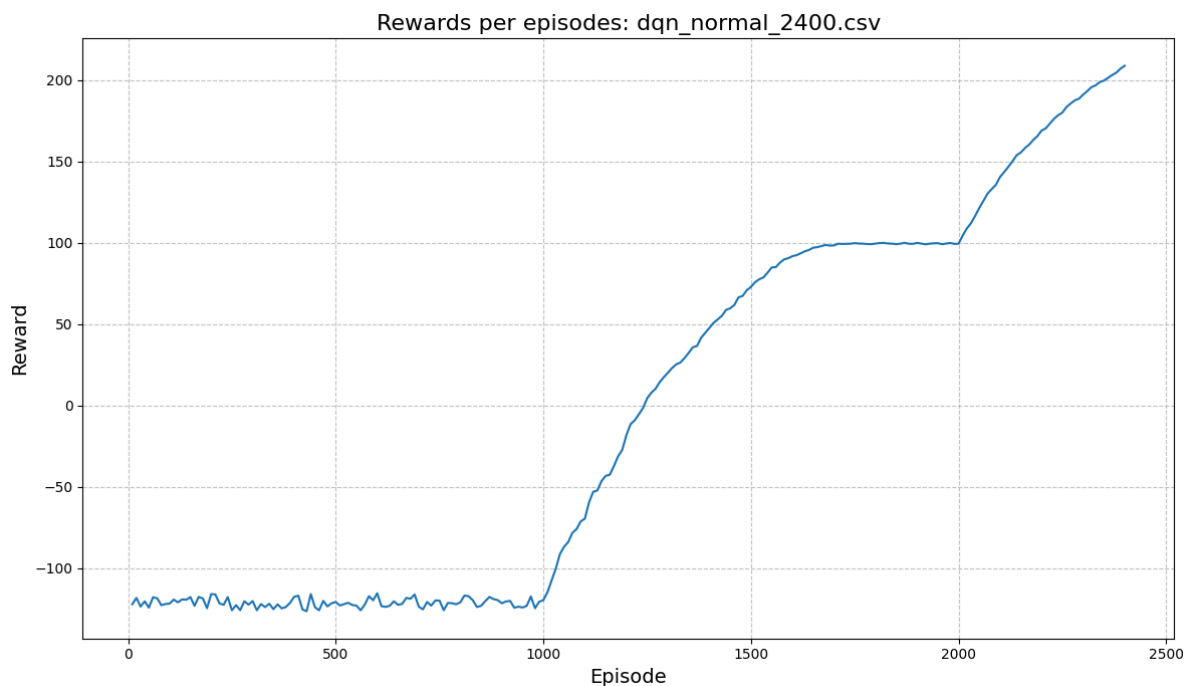
Résultats

En raison de la discrétisation de l'espace d'action, imposée par l'architecture du DQN, les mouvements des articulations du robot sont restés grossiers et binaires, de type "tout ou rien". Cette limitation empêche un contrôle fin des jambes, indispensable pour une marche fluide et équilibrée.

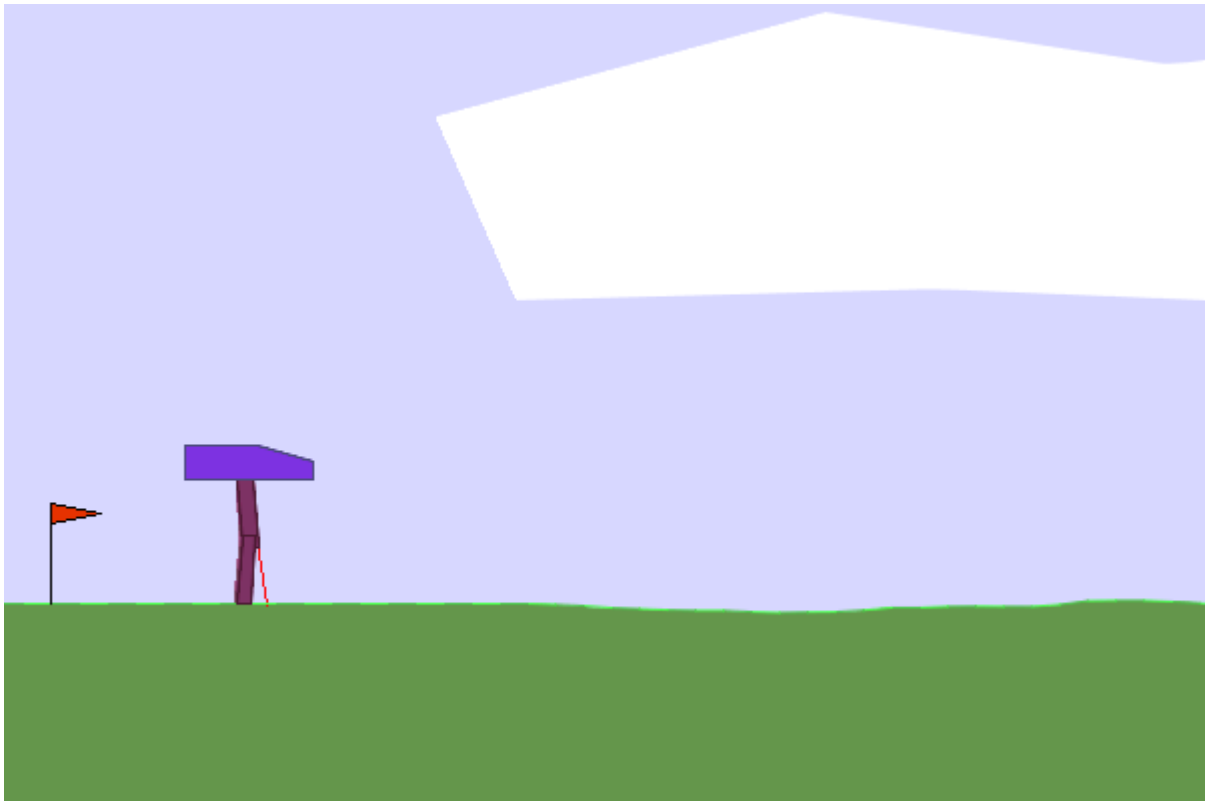
Ainsi, l'agent n'a jamais pu atteindre les 300 points requis (qui correspondent à l'arrivée à l'extrémité droite du terrain), mais a tout de même réussi à obtenir jusqu'à 200 points de récompense. Au fil de l'apprentissage, le robot a adopté une stratégie compensatoire : il utilisait presque exclusivement sa jambe avant pour se propulser, la positionnant en angle droit (rotation horaire à la hanche, rotation antihoraire au genou), tandis que la jambe arrière traînait au sol, freinant sa progression.

Durant les 1000 premiers épisodes sur les 2400 effectués, l'agent peinait à dépasser les -100 de récompense, chutant presque systématiquement sans réussir à marcher. Ce n'est qu'après cette phase que l'agent a commencé à comprendre partiellement comment avancer malgré les contraintes de son système d'action. Progressivement, ses performances se sont stabilisées, atteignant environ 200 points à l'épisode 2400.

J'ai interrompu l'entraînement à ce stade, estimant que la stratégie apprise, bien que optimale dans les limites du modèle DQN utilisé, ne permettrait pas d'atteindre les 300 points requis sans une architecture mieux adaptée aux actions continues, comme DDPG ou TD3.



Évolution des récompenses par épisodes du bipedal walker DQN sur terrain normal



10 premières secondes d'un épisode du bipedal walker DQN sur terrain normal

Twin Delayed Deep Deterministic Policy Gradient (TD3)

Face aux limites de la discrétisation de l'action dans le DQN, et insatisfait de m'arrêter à un robot qui plafonne à 200 points, j'ai exploré l'algorithme TD3 (Twin Delayed Deep Deterministic Policy Gradient), une évolution de DDPG, lui-même dérivé de DQN, conçue pour les environnements à actions continues comme BipedalWalker.

TD3 est un algorithme actor-critic qui apprend une politique déterministe (l'acteur) tout en évaluant la qualité des actions avec un ou plusieurs réseaux critiques. Il améliore DDPG en ajoutant trois mécanismes clés :

- Deux réseaux critiques (Q1 et Q2) pour limiter la surestimation de la valeur Q, en prenant le minimum entre les deux.
- Mise à jour retardée de l'acteur : l'acteur est mis à jour moins fréquemment que les critiques, pour plus de stabilité.
- Ajout de bruit à l'action cible pendant l'entraînement, pour améliorer l'exploration et éviter la convergence prématurée.

Avantages

- Supporte les actions continues, crucial pour contrôler les articulations du robot avec précision angulaire.

- Plus stable et performant que DDPG et DQN.
- Réduit la surestimation des valeurs d'action grâce aux deux critiques.

Inconvénients

- Algorithme complexe à implémenter manuellement.
- Exige des ressources computationnelles plus importantes (plus de réseaux, plus de données).

Pourquoi TD3 est mieux adapté à Bipedal Walker

Bipedal Walker nécessite un contrôle moteur fluide et précis des 4 articulations. TD3, avec ses sorties continues, permet de produire des mouvements plus naturels et adaptatifs, contrairement à DQN où les actions sont figées et binaires. Cette capacité permet au robot de mieux s'équilibrer, gérer les impacts, et progresser plus loin sur le terrain.

Implémentation via Stable-Baselines3

Étant donné la complexité de TD3, j'ai utilisé Stable-Baselines3, qui propose une implémentation accessible et efficace de TD3. Cette librairie offre également la fonctionnalité de reprise d'entraînement à partir d'un modèle existant, que j'ai exploitée pour :

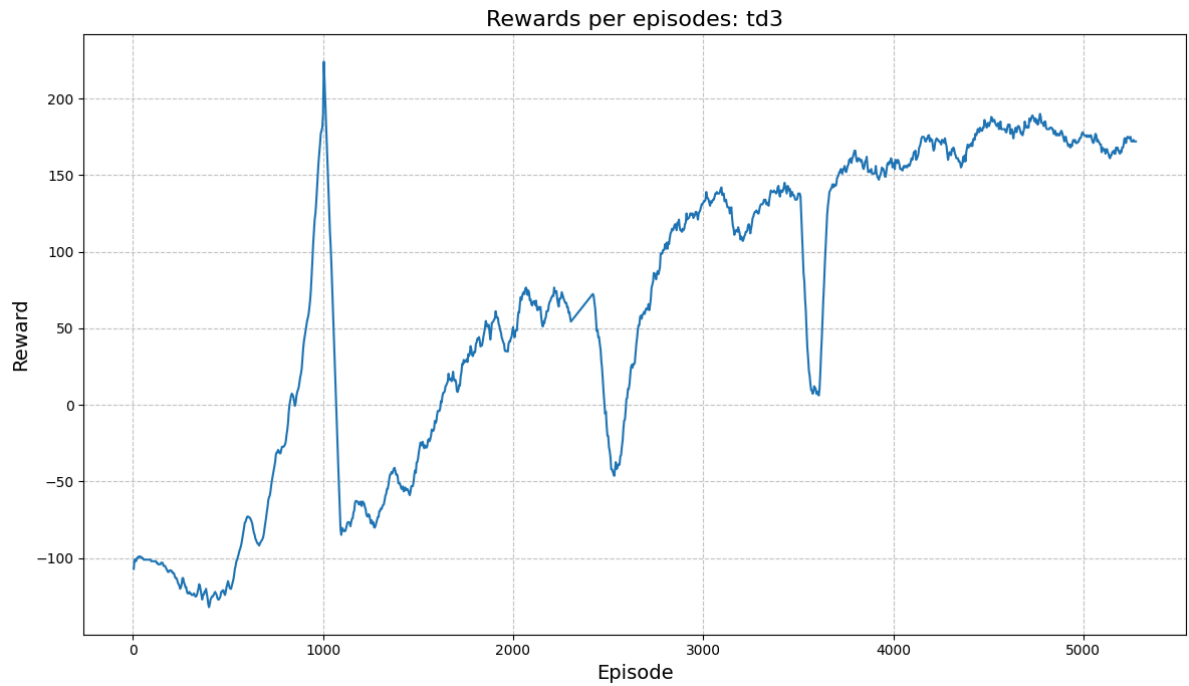
- Entraîner un premier modèle sur le terrain normal.
- Réutiliser ce modèle comme point de départ pour le terrain hardcore, bien plus exigeant.

Cette approche m'a permis de bénéficier du transfert d'apprentissage, facilitant l'adaptation du robot aux obstacles plus complexes du mode hardcore.

Résultats

L'entraînement sur terrain normal s'est déroulé de manière satisfaisante. Après une phase d'adaptation durant les 500 premiers épisodes, le robot a rapidement progressé, atteignant une récompense moyenne supérieure à 200 dès le 1000^e épisode, seuil auquel j'ai décidé d'arrêter l'entraînement. Le robot a adopté une stratégie originale : utiliser principalement sa jambe arrière pour la propulsion, tandis que la jambe avant restait en l'air, stabilisant son équilibre de manière élégante, presque comme une ballerine.

Ensuite, j'ai repris ce modèle entraîné sur terrain normal pour l'adapter au terrain hardcore. Sans surprise, les performances ont initialement chuté à environ -75 points de récompense, le robot n'ayant jamais été confronté aux fossés, marches et obstacles spécifiques au terrain difficile. Cependant, après environ 4000 épisodes d'adaptation, le robot a progressivement appris à gérer ces nouvelles contraintes, atteignant finalement une récompense moyenne de 175 sur terrain hardcore, démontrant une réelle capacité d'apprentissage et de transfert grâce à TD3.



Évolution des récompenses par épisodes du bipedal walker TD3 sur terrain normal jusqu'à 1000 épisode puis sur terrain hardcore



10 premières secondes d'un épisode du bipedal walker TD3 sur terrain hardcore

Taxi-V3

Présentation de l'environnement FrozenLake

L'environnement **Taxi-v3** simule un monde sous forme de **grille 5x5**, dans lequel un agent (le taxi) doit effectuer une mission de transport : **récupérer un passager à un point donné et le déposer à un autre point désigné**. Cette tâche implique la gestion de déplacements, la prise de décision séquentielle et une gestion efficace des ressources (temps, nombre d'actions).

But du jeu

Le taxi commence à une position aléatoire sur la grille. Le passager est lui aussi positionné aléatoirement dans l'un des quatre emplacements désignés : **Rouge (R)**, **Vert (G)**, **Jaune (Y)**, ou **Bleu (B)**. Il a également une destination cible, différente de sa position initiale. L'agent doit :

1. Se déplacer jusqu'au point de prise en charge,
2. Utiliser l'action **pickup** pour faire monter le passager à bord,
3. Se diriger vers le point de dépose,
4. Utiliser l'action **dropoff** pour finaliser le transport.

L'épisode se termine immédiatement après une dépose réussie du passager à sa destination.

Carte et représentation spatiale

La carte de la grille 5x5 comporte des murs qui bloquent certains déplacements. Les emplacements clés (R, G, Y, B) sont fixes.

Les déplacements dans certaines directions peuvent être empêchés par ces murs, ce qui rend la navigation non triviale.

Espace d'états

L'environnement possède **500 états discrets** combinant :

- **25 positions possibles du taxi** (5 lignes × 5 colonnes),
- **5 situations pour le passager** : dans l'un des 4 emplacements fixes ou déjà à bord du taxi,

- **4 destinations possibles** (R, G, Y, B).

Cela donne : $25 \times 5 \times 4 = 500$ états. Toutefois, seuls **404 états sont réellement atteignables**, car certains états sont invalides (comme quand le passager est déjà à destination).

Espace d'actions

Le taxi dispose de **6 actions discrètes** :

- 0** : se déplacer vers le **sud**
- 1** : se déplacer vers le **nord**
- 2** : se déplacer vers l'**est**
- 3** : se déplacer vers l'**ouest**
- 4** : **prendre** le passager (**pickup**)
- 5** : **déposer** le passager (**dropoff**)

Si une action est invalide (par exemple, essayer de prendre un passager absent, ou déposer au mauvais endroit), l'environnement renvoie une pénalité sans modifier l'état.

Récompenses

La fonction de récompense est simple mais punitive :

- +20** : pour une dépose correcte du passager,
- 10** : pour une tentative incorrecte de **pickup** ou **dropoff**,
- 1** : pour chaque action normale (déplacement ou autre action légale mais non concluante).

Cela encourage l'agent à optimiser le chemin et à éviter les erreurs de manipulation.

Déroulement d'un épisode

Un épisode débute dans un état valide (passager à un point, destination différente, passager hors du taxi). Il se termine lorsque :

- Le passager est correctement déposé à sa destination (**termination**),
- Ou qu'une limite de temps (par exemple 200 étapes) est atteinte (**truncation** si un *time limit wrapper* est utilisé).

Propriétés de l'environnement

- **Déterministe** : à chaque action correspond toujours le même résultat (pas de stochasticité).
- **Facile à simuler** : ne nécessite pas de modélisation externe de l'environnement.

Q learning

Pour l'environnement Taxi-v3, j'ai implémenté l'algorithme de Q-Learning, une méthode d'apprentissage par renforcement hors-modèle et hors politique. Le principe repose sur la construction d'une table Q-table qui associe à chaque paire état-action une estimation de la récompense attendue, mise à jour à chaque interaction avec l'environnement selon la formule de Bellman. Le but est d'apprendre une politique optimale en maximisant les valeurs Q.

Cet algorithme est particulièrement adapté à Taxi-v3, car cet environnement possède un espace d'états et d'actions discret et de taille gérable, ce qui permet de stocker efficacement la Q-table en mémoire. De plus, la dynamique de l'environnement (déplacements, ramassage et dépôt de passagers) est déterministe, ce qui facilite la convergence de Q-Learning.

Avantages

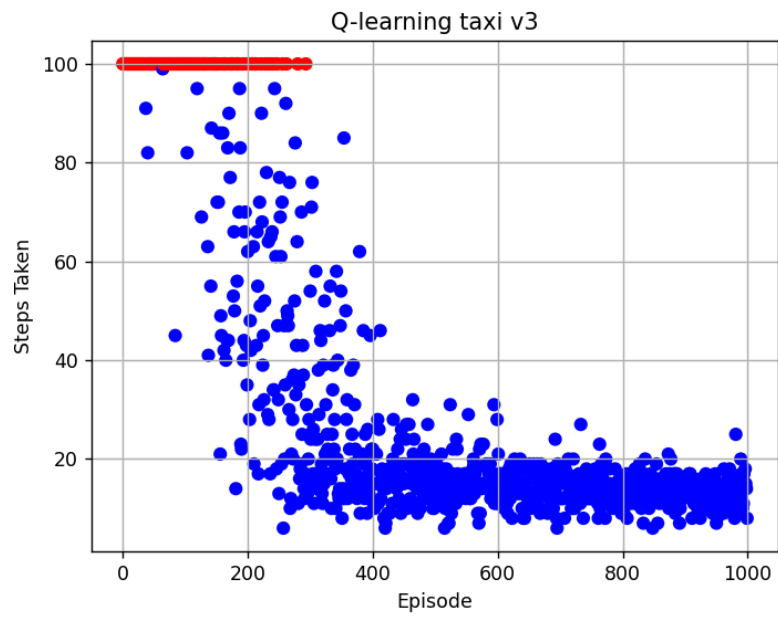
- Simple à implémenter,
- Converge rapidement sur des petits environnements discrets,
- Donne de bons résultats sans architecture de réseau neuronal.

Inconvénients

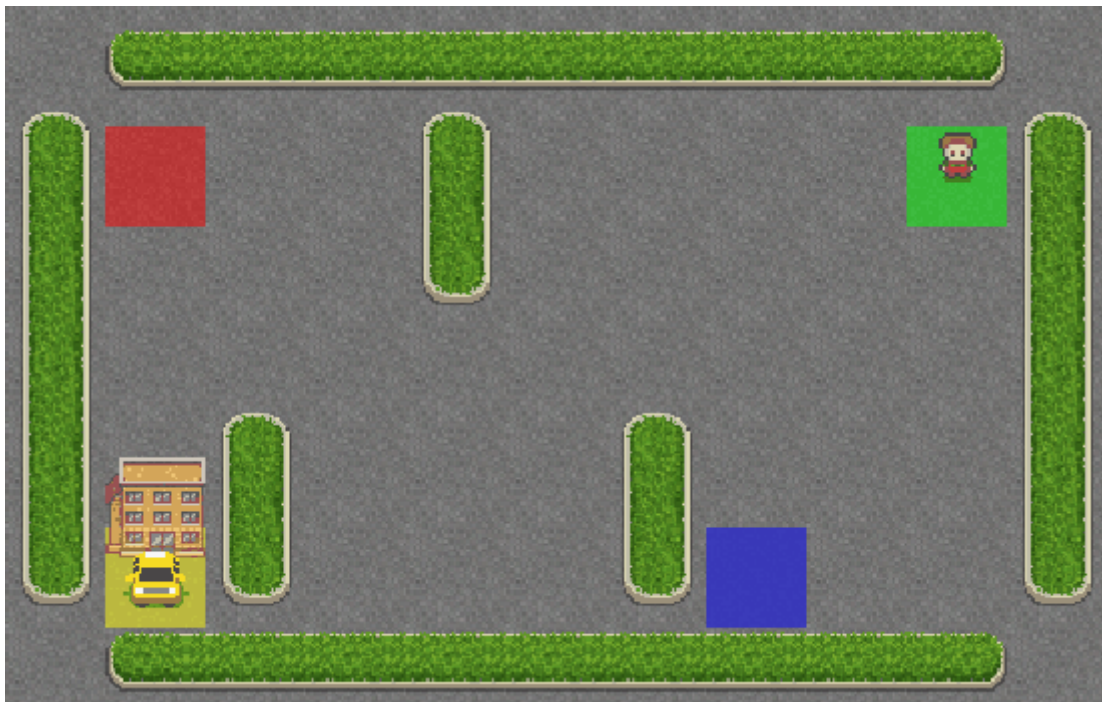
- Ne gère pas les espaces d'états ou d'actions continus ou très vastes (ce qui n'est pas le cas de Taxi-v3),
- Nécessite une table Q en mémoire, dont la taille augmente rapidement avec la complexité de l'environnement (cela ne pose problème qu'en cas de carte customisée gigantesque).

Résultats

Dans le cas de Taxi-v3, Q-Learning est donc une solution parfaitement adaptée, offrant un bon compromis entre simplicité, efficacité et performance.



Réduction du nombre d'étapes prises au fil de 1000 épisodes d'entraînement



Exécution d'un épisode de Taxi-v3 sur carte par défaut après 10000 épisodes

Monte Carlo

Méthode de Monte Carlo

Pour l'environnement Taxi-v3, j'ai implémenté une approche d'apprentissage par renforcement basée sur la méthode **Monte Carlo**, qui repose sur l'estimation des valeurs d'action à partir d'épisodes complets. Contrairement au Q-Learning, Monte Carlo ne met pas à jour les valeurs de manière incrémentale à chaque étape, mais attend la fin de l'épisode pour calculer le **retour total** (cumul des récompenses actualisées) de chaque paire état-action rencontrée.

J'ai utilisé la version **First-Visit Monte Carlo**, qui ne met à jour les valeurs que lors de la première apparition de chaque paire (état, action) dans un épisode. Les retours sont stockés et moyennés pour chaque paire, ce qui permet de construire progressivement une estimation de la Q-table. Le choix des actions suit une stratégie **ϵ -greedy** avec un taux d'exploration décroissant au fil des épisodes, favorisant l'exploitation des connaissances acquises.

Même si Taxi-v3 est un environnement simple avec un espace d'états discret et de taille modérée, la méthode Monte Carlo montre certaines limites dans ce contexte.

Avantages

- Approche conceptuellement simple, ne nécessitant pas de modèle de l'environnement.
- Ne dépend pas de la fonction de transition : aucune hypothèse sur la dynamique de l'environnement.
- Utile pour apprendre directement à partir de trajectoires complètes, notamment en environnement simulé.

Inconvénients

- Apprentissage plus lent : les mises à jour sont faites uniquement à la fin des épisodes.
- Moins stable dans les environnements stochastiques ou avec des épisodes longs.
- Moins efficace dans Taxi-v3 que Q-Learning, car ce dernier bénéficie d'un retour immédiat sur chaque interaction, accélérant la convergence.

Résultats

Dans mon implémentation, la méthode Monte Carlo a donné des résultats moins satisfaisants que Q-Learning. Malgré un apprentissage sur 30 000 épisodes, la politique apprise ne converge pas aussi rapidement, et les performances restent variables, notamment à cause de la dépendance au retour global d'un épisode. Cette méthode reste néanmoins intéressante à étudier pour des environnements où la dynamique est inconnue ou non déterministe, ou encore lorsqu'on dispose uniquement de trajectoires complètes hors ligne (off-policy).

CarRacing-v3

L'objectif de ce projet est d'entraîner un agent à jouer au jeu CarRacing-v3 en utilisant l'algorithme de Deep Q-Learning (DQN). L'environnement est fourni par la bibliothèque Gymnasium, et les images du jeu servent d'observations pour guider les décisions de l'agent.

Dans CarRacing-v3, l'apprentissage est basé uniquement sur les images de la scène. Pour rendre ces images plus exploitables par le modèle, plusieurs transformations ont été appliquées. La première consiste à convertir chaque image en niveaux de gris (gray-scale), ce qui permet de ne conserver qu'une seule valeur d'intensité lumineuse par pixel, allant de 0 à 255. Ensuite, une réduction de la taille de l'image à 84 x 84 pixels est réalisée. Cette réduction conserve les éléments visuels essentiels tout en diminuant la complexité computationnelle, ce qui accélère l'entraînement. Enfin, un cropping (recadrage) est appliqué afin de supprimer la partie inférieure de l'image contenant les contrôles visibles à l'utilisateur, mais inutiles pour l'agent.

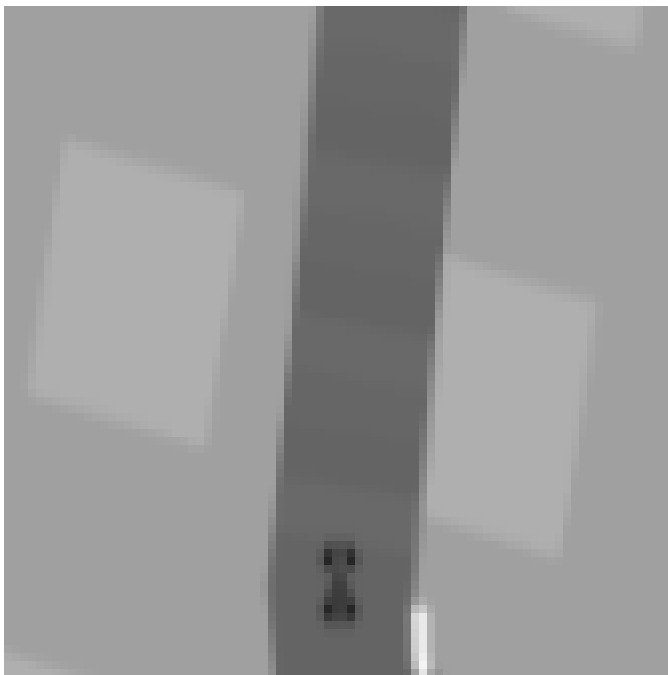


Image après transformations

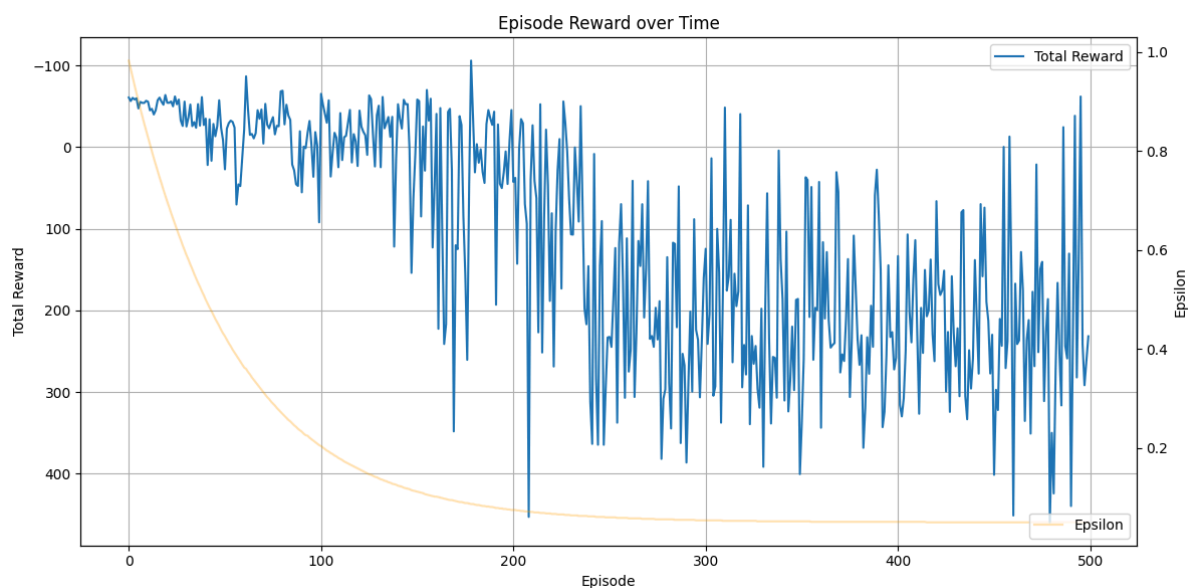
L'agent est piloté par un réseau de neurones appelé Q-network, qui estime la valeur des actions possibles dans un état donné. Un second réseau, appelé target network, est utilisé en parallèle pour stabiliser l'apprentissage. Ces deux réseaux sont régulièrement synchronisés pendant l'entraînement.

Pour améliorer l'efficacité de l'apprentissage, une mémoire de rejouabilité (Replay Buffer) est mise en place. Elle stocke les expériences passées de l'agent (état, action, récompense, nouvel état, fin de l'épisode), permettant ainsi de réentraîner le modèle sur des situations diverses sans dépendre uniquement de l'ordre chronologique des actions.

Une stratégie d'exploration est utilisée : l'agent ne choisit pas toujours l'action la plus avantageuse selon lui, mais explore aussi aléatoirement de nouvelles options. Cette exploration diminue progressivement au fil des épisodes afin de favoriser l'exploitation des connaissances acquises.

Les observations utilisées par le modèle sont en fait des séquences de quatre images consécutives empilées. Cette technique permet de capturer la dynamique du mouvement, ce qui est essentiel dans un jeu de course comme CarRacing.

L'apprentissage s'effectue sur plusieurs épisodes, pendant lesquels l'agent interagit avec l'environnement, stocke ses expériences, s'entraîne par échantillonnage aléatoire de la mémoire, et ajuste son réseau de neurones. Le modèle est régulièrement sauvegardé afin de pouvoir être réutilisé ou affiné ultérieurement.



Ce graphique montre l'évolution de la récompense totale obtenue par l'agent à chaque épisode, ainsi que la valeur d'épsilon utilisée dans la stratégie d'exploration. On observe une forte variabilité des performances, typique des environnements complexes comme CarRacing. La courbe décroissante de l'épsilon (en beige) illustre la réduction progressive de l'exploration au profit de l'exploitation. Malgré l'instabilité, certains épisodes montrent des performances plus élevées, indiquant que l'agent parvient parfois à adopter un comportement efficace.

Frozen lake

Présentation de l'environnement FrozenLake

FrozenLake est un environnement d'apprentissage par renforcement proposé dans la bibliothèque Gymnasium. Il consiste en une grille représentant un lac gelé que l'agent doit traverser pour atteindre un objectif, en évitant de tomber dans des trous (holes). Le point de départ est toujours la case en haut à gauche ([0, 0]) et la case but est située en bas à droite ([3, 3] pour une carte 4x4 par défaut). L'environnement est **discret**, avec un espace d'états de 16 positions (pour la grille 4x4) et un espace d'actions composé de 4 directions possibles : gauche, bas, droite et haut.

Une particularité du jeu réside dans la **glissance du sol** (paramètre `is_slippery=True` par défaut), qui introduit de la stochasticité dans les déplacements : l'agent peut ne pas aller exactement dans la direction prévue. L'agent reçoit une récompense de 1 en atteignant la case but, et 0 dans tous les autres cas. L'épisode se termine soit lorsque l'agent tombe dans un trou, soit lorsqu'il atteint l'objectif.

Q learning

Comme pour le taxi-v3, nous avons utilisé le Q-Learning sur l'environnement FrozenLake.

Q-Learning fonctionne ici en apprenant progressivement les meilleures actions à prendre dans chaque état en mettant à jour la Q-table via la formule de Bellman, basée sur l'expérience de l'agent et les récompenses reçues. L'algorithme explore l'environnement tout en exploitant les connaissances acquises, ce qui lui permet de converger vers une politique optimale.

Avantages

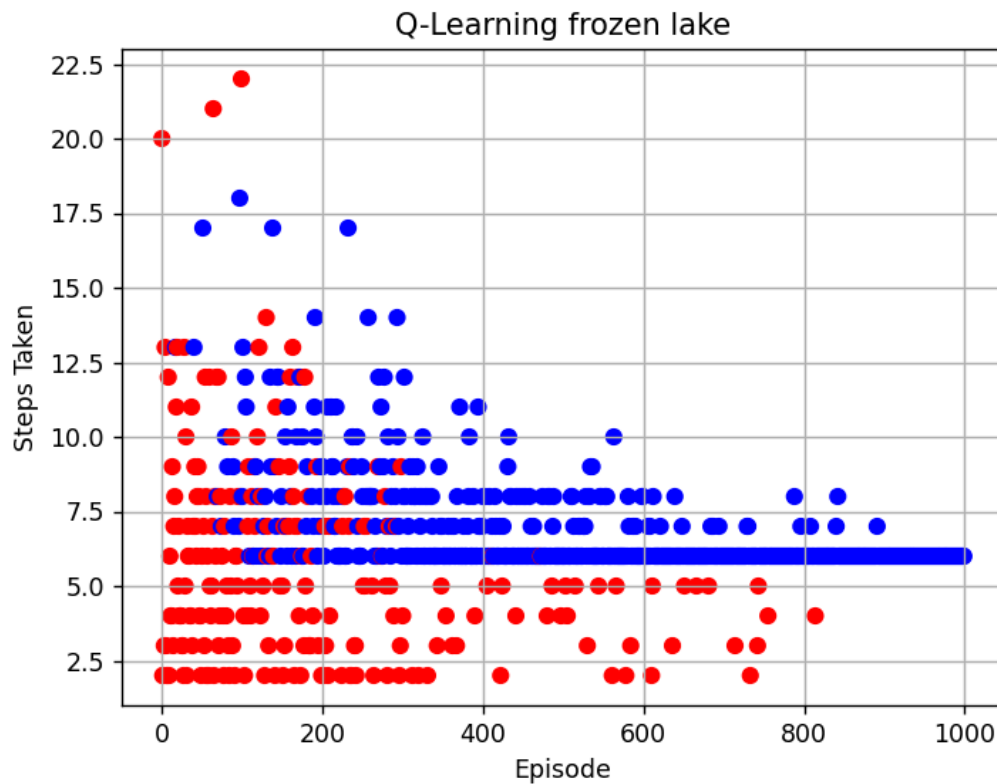
- Facile à implémenter et à comprendre,
- Idéal pour les environnements discrets de petite taille comme FrozenLake,
- Permet de gérer l'exploration/exploitation efficacement via une stratégie epsilon-greedy.

Inconvénients

- Peu efficace sur les environnements stochastiques, comme FrozenLake avec `is_slippery=True`, où les actions peuvent ne pas produire toujours le même résultat.

Résultats

Malgré la stochasticité du terrain glissant, Q-Learning reste une méthode pertinente pour FrozenLake, permettant d'apprendre une stratégie raisonnable avec un bon taux de réussite après suffisamment d'épisodes d'apprentissage.



Réduction du nombre d'étapes prises au fil de 1000 épisodes d'entraînement



Exécution d'un épisode de Frozen lake sur carte par défaut après 10000 épisodes

Monte Carlo

Comme pour le Taxi Driver mentionné précédemment, j'ai expérimenté l'apprentissage par renforcement via la méthode **Monte Carlo** sur l'environnement **FrozenLake**, où l'objectif est d'amener l'agent du point de départ à la case d'arrivée sans tomber dans un trou. Chaque épisode est une trajectoire complète sur la grille, avec des récompenses distribuées uniquement à la fin (1 en cas de succès, 0 sinon).

J'ai utilisé une version **First-Visit Monte Carlo**, qui met à jour la Q-table uniquement lors de la première visite d'une paire (état, action) dans un épisode. À la fin de chaque épisode, les retours (rewards cumulés actualisés) sont calculés pour chaque étape, et la valeur de chaque action est estimée par la **moyenne des retours observés**. Une stratégie **ϵ -greedy** permet d'équilibrer exploration et exploitation, avec une décroissance progressive d' ϵ pour favoriser la convergence vers une politique optimale.

L'environnement par défaut de FrozenLake étant **déterministe** grâce au paramètre (`is_slippery=False`), la méthode Monte Carlo s'y adapte particulièrement bien, car les transitions sont prévisibles et les épisodes suffisamment courts pour que l'évaluation des retours soit efficace.

Avantages

- Ne nécessite pas de modélisation de l'environnement.
- Idéal pour les environnements **déterministes** avec des épisodes de durée raisonnable.
- Convergence stable grâce à l'échantillonnage complet des trajectoires.
- Facile à implémenter dans les environnements discrets comme FrozenLake.

Inconvénients

- Mise à jour lente : les Q-valeurs ne sont mises à jour qu'à la fin des épisodes.
- Moins efficace si les épisodes sont longs ou les trajectoires très variables.
- Moins performant si l'environnement est stochastique (`is_slippery=True`), car les résultats des actions deviennent imprévisibles.

Résultats

Sur la version **déterministe** de FrozenLake, la méthode Monte Carlo a montré une excellente capacité d'apprentissage. Après environ **10 000 épisodes**, l'agent réussit régulièrement à atteindre l'objectif en suivant une stratégie stable et efficace. Cela démontre que cette méthode peut constituer une **alternative viable** au Q-Learning dans des environnements simples, discrets et peu bruités

Lunar Lander

LunarLander est un environnement de simulation 2D dans lequel un module spatial doit se poser délicatement sur une plateforme située entre deux drapeau. Le module est équipé d'un moteur principal orienté vers le bas et de deux moteurs latéraux permettant de s'orienter à gauche ou à droite.

Il comporte 8 variables d'état (position, vitesse, angle, contact au sol...) et 4 actions possibles (ne rien faire, moteur principal, moteur gauche, moteur droit).

Q-Learning

J'ai d'abord expérimenté la méthode classique du Q-Learning, une approche simple du renforcement qui repose sur une Q-table pour apprendre la valeur de chaque action dans chaque état. Cependant, cette méthode est rapidement limitée dans des environnements à espace d'état continu comme LunarLander.

En effet, Q-Learning suppose que l'on peut représenter tous les états possibles dans une table, ce qui devient impraticable quand les états sont décrits par des valeurs continues (comme ici avec les positions, vitesses ou angles).

Avantages

- Rapidité d'entraînement

Inconvénients

- Difficulté à représenter efficacement tous les états dans une Q-table,
- Comportement aléatoire et peu efficace, même après de nombreux épisodes

Résultats

Après plus de 1500 épisodes, le vaisseau ne se pose malheureusement pas, sauf cas très exceptionnel qui tient davantage du hasard que de l'entraînement à proprement parler. Pour cette raison, on peut convenir que cet algorithme d'entraînement n'est pas adapté à cet environnement.

Deep Q-Learning

J'ai ensuite appliqué le Deep Q-Learning en raison de son adaptation aux environnements continue. Comme on l'a vu précédemment, il est compliqué d'utiliser une Q-table dans ce genre d'environnement. Contrairement au Q-Learning, le Deep Q-Learning utilise un réseau de neurones pour approximer la fonction Q, c'est-à-dire estimer la valeur d'une action dans un état donné. Le réseau prend en entrée les 8

paramètres de l'environnement et donne en sortie la valeur estimée (Q-value) de chaque action.

Le modèle apprend à choisir les bonnes actions en stockant des expériences dans une mémoire tampon, puis en s'entraînant régulièrement sur ces expériences aléatoires pour stabiliser l'apprentissage. Une stratégie epsilon-greedy est utilisée pour équilibrer exploration (essayer de nouvelles actions) et exploitation (choisir les meilleures actions connues).

Avantages

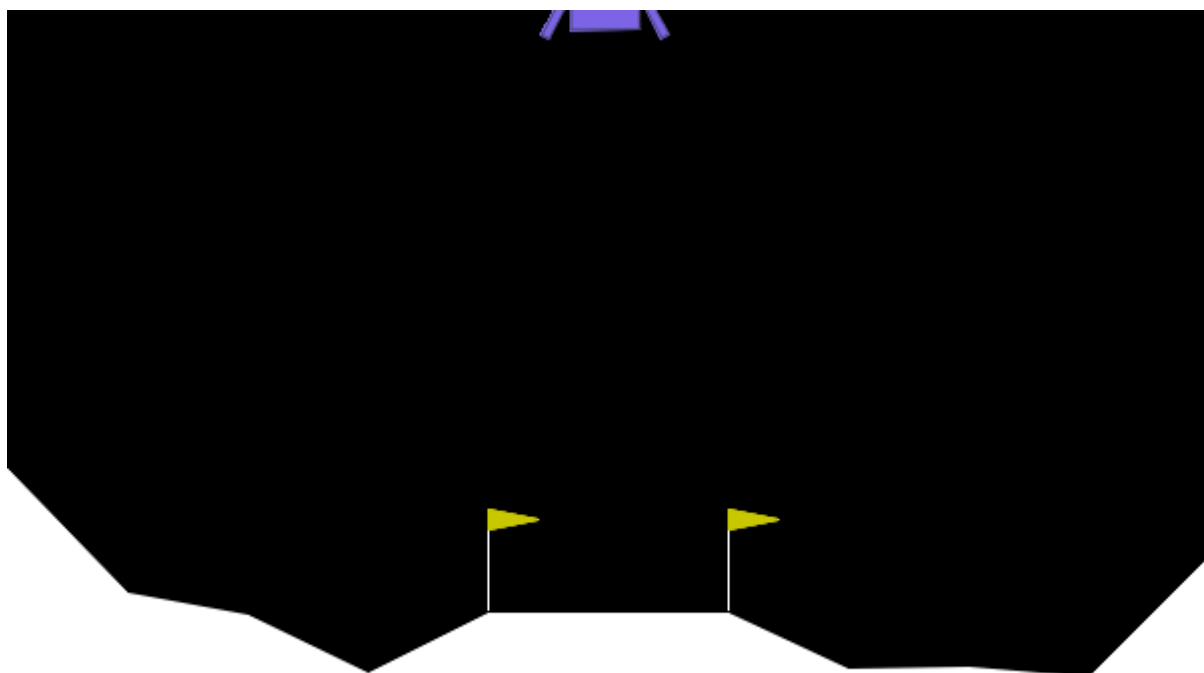
- Adapté aux environnements à états continus comme LunarLander, là où une Q-table ne suffirait plus,
- Capable d'apprendre des politiques complexes grâce à un réseau de neurones,
- Permet de s'adapter à des situations non vues exactement pendant l'entraînement.

Inconvénients

- Plus complexe à implémenter et à régler (réseau, taux d'apprentissage, epsilon, etc.),
- Besoin de beaucoup plus d'épisodes d'entraînement pour converger,

Résultats

Après un nombre suffisant d'épisodes (À partir de 700 les résultats commencent à être intéressants), l'agent apprend à atterrir proprement sur la plateforme, en contrôlant sa descente avec précision. Le score augmente progressivement et peut dépasser les 200 points, indiquant un atterrissage réussi et maîtrisé. DQN s'avère donc particulièrement adapté à cet environnement, à condition de disposer d'un bon réglage et d'un temps d'entraînement suffisant.



Conclusion

Ce projet a permis d'explorer l'efficacité de différents algorithmes d'apprentissage par renforcement sur une variété d'environnements Gymnasium, révélant l'importance cruciale de l'adéquation entre algorithme et type d'environnement.

Synthèse des résultats

Environnements fixes (Taxi-v3, Frozen Lake) : Le Q-Learning classique s'est révélé optimal avec une convergence rapide et stable. Monte Carlo, bien que fonctionnel, présente des performances moindres due à ses mises à jour différées.

Environnements continus (Lunar Lander) : Le Deep Q-Learning a démontré sa pertinence, maîtrisant l'atterrissage après 700 épisodes grâce à son réseau de neurones qui approxime la fonction Q. À l'inverse, le Q-Learning se révèle complètement inefficace dû à son environnement continu qui empêche la création d'une Q-table suffisamment importante pour répondre à tous les cas de figure.

Environnements continus complexes (Bipedal Walker, Car Racing) : Pour ces environnements, on se rend compte que le DQL a du mal car il ne peut faire que des mouvements basiques "marche/arrêt". On a des résultats plus probants grâce au TD3 permet de varier l'intensité des mouvements, donnant de bien meilleurs résultats.

Enseignements clés

L'adéquation algorithme-environnement détermine la réussite d'un projet. Utiliser l'algorithme le plus sophistiqué, généralement le plus gourmand en ressources, n'est pas toujours le meilleur choix. Ce projet démontre l'importance de sélectionner l'algorithme en fonction des spécificités de l'environnement et des objectifs visés : un Q-Learning simple peut surpasser un DQN complexe sur des jeux à choix limités, tandis que TD3 devient indispensable pour les environnements qui changent tout le temps.